

Constraint Programming Model for the Minimum Consistent Finite Automaton Problem

Virginia Nicosia

April 16, 2025

1 Introduction

A Deterministic Finite Automaton (DFA) is a model of computation defined by a set of states, an alphabet (here $\{0, 1\}$), a transition function, an initial state, and a set of accepting states. A sequence of symbols is accepted if, starting from the initial state and following the transitions dictated by the sequence symbols, the automaton ends in an accepting state otherwise the sequence is rejected.

The Minimum Consistent Finite Automaton (MCFA) problem asks for a DFA with the minimum possible number of states that is consistent with a given sample of sequences. This sample consists of two disjoint sets: a set A of sequences that the DFA must accept, and a set R of sequences that the DFA must reject. The challenge is to ensure the solution is both minimal and consistent with the sample.

This report describes a Constraint Programming model, implemented using the Gecode library, to solve the MCFA problem.

2 Constraint Programming Model

The number of states n is not fixed a priori; the approach iterates through possible values of n , starting from 1, until a consistent automaton is found.

2.1 Variables

Let n be the number of states in the DFA, indexed from 0 to $n - 1$, with state 0 is designated as the initial state. The model uses the following decision variables:

- **Transitions (transitions):** An integer variable array of size $2n$. For each state $i \in \{0, \dots, n - 1\}$ and each input symbol $b \in \{0, 1\}$, `transitions[2i+b]` represents the index of the next state reached from state i upon reading symbol b . The domain of each variable is $\{0, \dots, n - 1\}$.
- **Accepting States (accepting):** A boolean variable array of size n . For each state $i \in \{0, \dots, n - 1\}$, `accepting[i]` is 1 if state i is an accepting state, and 0 otherwise. The domain is $\{0, 1\}$.

2.2 Constraints

Constraints are imposed to ensure that the resulting variable assignment corresponds to a DFA that correctly classifies all sequences in the sets A and R .

1. **Acceptance Constraints:** For each sequence $w = (w_0, w_1, \dots, w_{m-1}) \in A$:

- We simulate the path of the automaton starting from the initial state (state 0). Let $s_0 = 0$. For k from 0 to $m - 1$, the next state s_{k+1} is determined by the transition from state s_k on input w_k . This is modeled using Gecode's `element` constraint, which allows dynamic indexing into the transition array: $s_{k+1} = \text{transitions}[2s_k + w_k]$.
- After processing the entire sequence, the automaton is in the final state s_m . This final state must be an accepting state. This is enforced by the constraint `accepting[sm] = 1`. In the Gecode implementation, this is expressed using implication: $(s_m = i) \implies (\text{accepting}[i] = 1)$ for all $i \in \{0, \dots, n - 1\}$.

2. **Rejection Constraints:** For each sequence $w = (w_0, w_1, \dots, w_{m-1}) \in R$:

- Similar to acceptance, we simulate the path of the automaton starting from state 0 to find the final state s_m after processing w , using the `element` constraint for transitions.
- The final state s_m must *not* be an accepting state. This is enforced by the constraint `accepting[sm] = 0`. Using implication: $(s_m = i) \implies (\text{accepting}[i] = 0)$ for all $i \in \{0, \dots, n-1\}$.

3. **Symmetry Breaking/Ordering:** The constraint `transitions[2i] ≤ transitions[2i + 1]` for all states i , does not change the set of representable automata, it serves as a symmetry-breaking constraint. Multiple DFAs can be equivalent in behavior but differ structurally; imposing such constraints can reduce the number of symmetric solutions the solver needs to explore, potentially helping it converge more efficiently.

2.3 Branching Strategy

Branching strategies are defined for both transition and acceptance variables to guide the solver during the search. The strategy used prioritizes assigning variables with smaller current domains first (`INT_VAR_SIZE_MIN`) and attempts smaller values first (`INT_VAL_MIN`, `BOOL_VAL_MIN`). This guides the search towards potentially simpler solutions earlier.

3 Search Strategy and the Role of DFS

To find the automaton with the *minimum* number of states, I used an iterative approach combined with a specific search algorithm.

3.1 Iterative Minimization

The program iterates through possible numbers of states n , starting from $n = 1$ and increasing ($n = 1, 2, 3, \dots$); for each n , it creates an instance of the CP model described above (the ‘MCFA’ space in the code) and attempts to find a solution.

3.2 Depth-First Search (DFS)

For a fixed number of states n , the Gecode library’s Depth-First Search (DFS) engine is used to explore the solution space. In the code:

```
DFS<MCFA> engine(model);
```

DFS is employed for the following reasons:

- **Systematic Exploration:** DFS systematically explores the search tree generated by the branching strategy.
- **Finding the First Solution:** The code calls `engine.next()`, which returns the *first* solution found by the DFS engine for the current number of states n . Since the goal is the minimum n , finding any valid solution for the current n is sufficient.
- **Guaranteeing Minimality:** Because the program iterates on n starting from 1, the very first solution found across all iterations of n corresponds to the smallest possible number of states for which a consistent automaton exists. As soon as a solution is found for a particular n , the program terminates and prints that solution. This ensures that the output automaton has the minimum number of states required by the problem definition. If DFS explores the entire search space for a given n without finding a solution, the program proceeds to try $n + 1$.

In essence, DFS is used here as a complete search algorithm to find a feasible solution for a fixed n , and the minimality is guaranteed by the outer loop that incrementally increases n .

4 Implementation Details

The model is implemented as a Gecode ‘Space’ subclass named ‘MCFA’.

- Integer variables and Boolean variables from Gecode represent the state transitions and accepting properties, respectively.
- Relational constraints (rel), element constraints (element), and implication ($>>$ operator) are used to model the transition logic and the acceptance/rejection conditions based on the final state reached for each sequence.
- The search is driven by the standard ‘DFS’ engine provided by Gecode, configured with the branching strategy mentioned earlier.

5 Conclusion

By defining variables for the automaton’s structure (transitions, accepting states) and constraints reflecting the required behavior on sample sequences, a CP solver can search for a valid configuration for the Minimum Consistent Finite Automaton problem.

The combination of iterating through the number of states n (starting from 1) and using Depth-First Search to find the first consistent solution for a given n effectively guarantees finding an automaton with the minimum required number of states. The resulting solutions are output in a format compatible with the provided checker tool, allowing for validation of correctness and generation of visual diagrams of the minimal DFA found.